

Московский Физико-Технический Институт

Отчёт по эксперименту

Лабораторная по контейнерам

Выполнил:
Студент 1 курса ФАКТ
Группа Б03-504
Подмосковнов Лев

1 Цель работы

Цель лабораторной работы — экспериментально исследовать время выполнения основных операций над контейнерами языка C++ и сравнить стандартные контейнеры STL с пользовательскими реализациями.

В работе рассматриваются следующие операции:

1. добавление элемента в конец вектора;
2. вставка элемента в произвольное место вектора;
3. удаление элемента из произвольного места вектора;
4. добавление элемента в начало односвязного списка;
5. удаление элемента из начала односвязного списка;
6. добавление элемента в бинарное дерево;
7. обход всего контейнера.

2 Эксперимент 0. Добавление в конец вектора

2.1 Описание эксперимента

В данном эксперименте исследуется изменение значений `size` и `capacity` контейнера `std::vector` при последовательном добавлении элементов с помощью `push_back`.

`size` показывает фактическое количество элементов в векторе, а `capacity` — количество элементов, которое может быть размещено в уже выделенной памяти без реаллокации.

2.2 График

На рисунке 1 должен быть представлен график зависимости `size` и `capacity` от номера итерации.

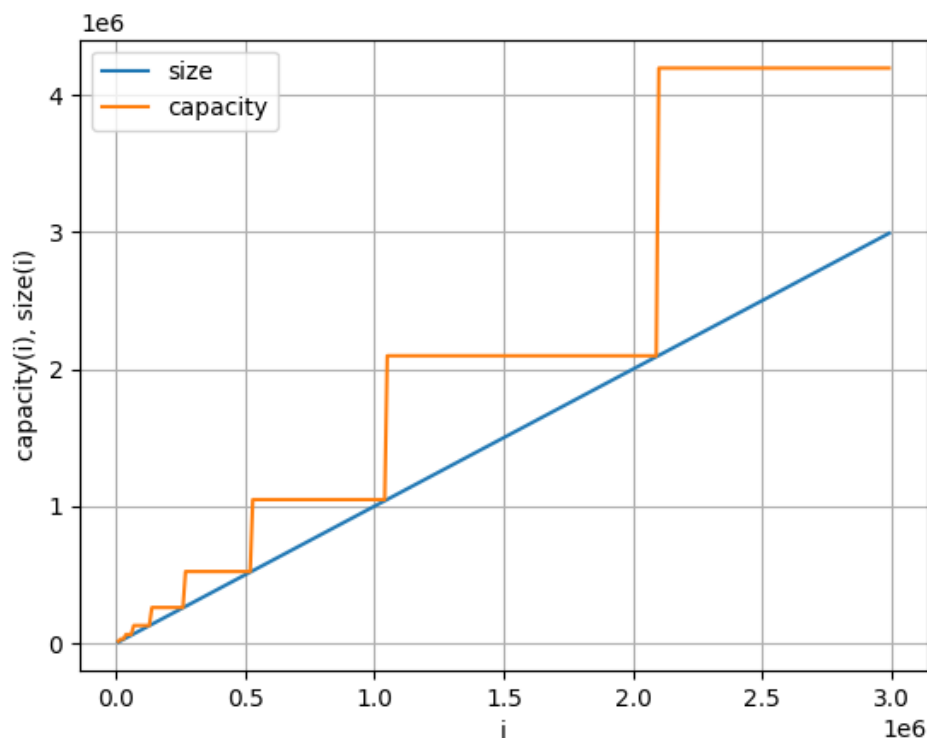


Рис. 1: Изменение `size` и `capacity` при выполнении `push_back`

2.3 Вывод

Значение `size` увеличивается на единицу при каждом добавлении элемента. Значение `capacity` увеличивается скачками. Это связано с тем, что вектор заранее выделяет память с запасом, чтобы не выполнять реаллокацию при каждом вызове `push_back`.

3 Эксперимент 1. Вставка элемента в произвольное место вектора

3.1 Описание эксперимента

В этом эксперименте сравнивается среднее время вставки элемента в произвольное место для стандартного контейнера `std::vector` и пользовательского контейнера `subvector`.

Для стандартного вектора используется метод `insert`. Для пользовательского контейнера должна быть реализована аналогичная операция.

3.2 Идея реализации

Для вставки элемента в произвольную позицию необходимо:

1. при необходимости увеличить ёмкость массива;
2. сдвинуть все элементы справа от позиции вставки на одну ячейку вправо;
3. записать новый элемент в освободившееся место;
4. увеличить размер контейнера.

3.3 График

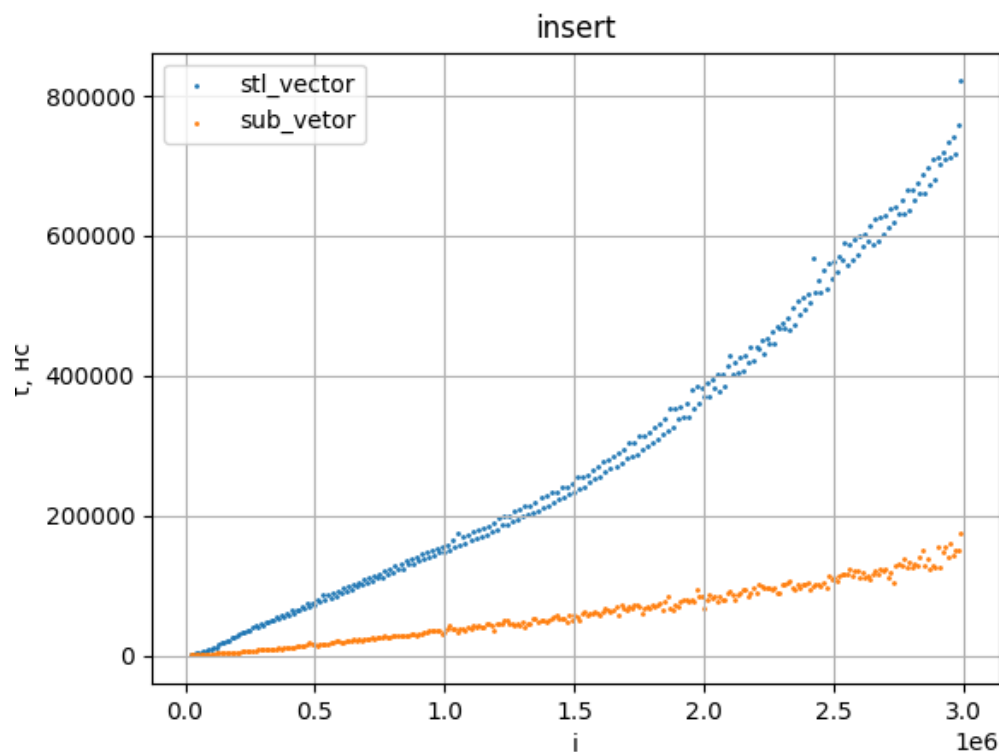


Рис. 2: Среднее время вставки элемента в произвольное место вектора

3.4 Асимптотика

При вставке в произвольную позицию в среднем требуется сдвинуть порядка $\frac{n}{2}$ элементов. Поэтому средняя сложность операции:

$$T(n) = O(n).$$

4 Эксперимент 2. Удаление элемента из произвольного места вектора

4.1 Описание эксперимента

В данном эксперименте сравнивается среднее время удаления элемента из произвольного места для `std::vector` и пользовательского контейнера `subvector`.

Для стандартного контейнера используется метод `erase`. Для пользовательского контейнера должна быть реализована аналогичная операция.

4.2 Идея реализации

Для удаления элемента из произвольной позиции необходимо:

1. выбрать позицию удаляемого элемента;
2. сдвинуть все элементы справа от этой позиции на одну ячейку влево;
3. уменьшить размер контейнера.

4.3 График

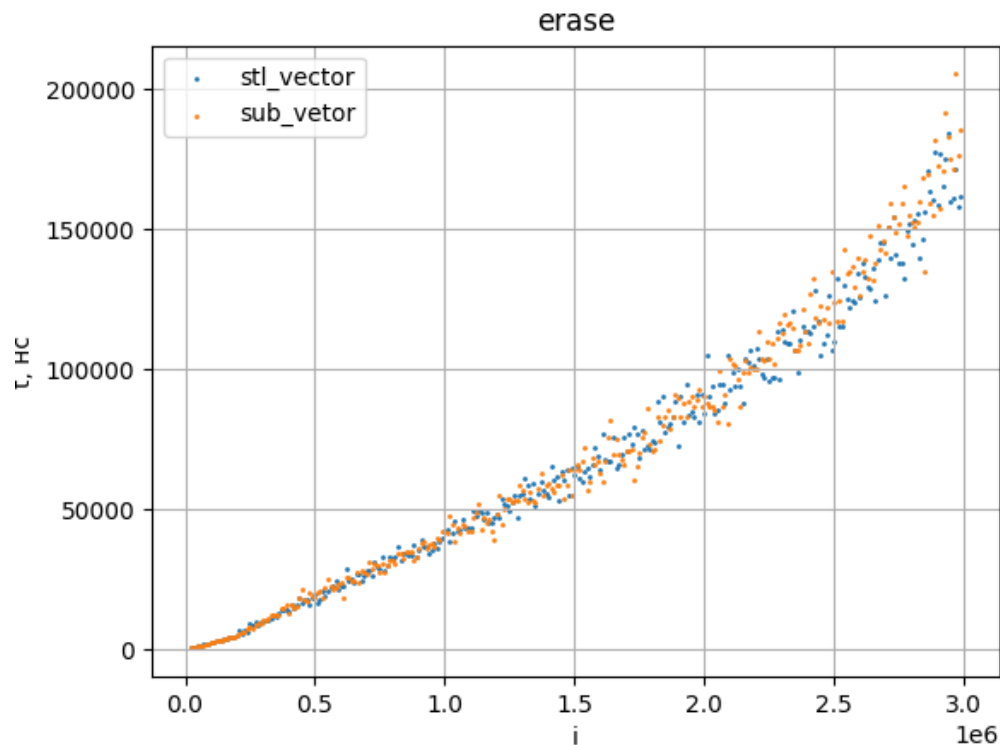


Рис. 3: Среднее время удаления элемента из произвольного места вектора

4.4 Асимптотика

При удалении из произвольной позиции в среднем требуется сдвинуть порядка $\frac{n}{2}$ элементов. Поэтому средняя сложность операции:

$$T(n) = O(n).$$

5 Эксперимент 3. Добавление элемента в начало односвязного списка

5.1 Описание эксперимента

В данном эксперименте измеряется среднее время работы операции добавления элемента в начало списка для контейнеров:

- `std::list`;
- `std::forward_list`;
- пользовательского `subforward_list`.

Для `std::list` и `std::forward_list` используется метод `push_front`. Для пользовательского контейнера используется метод `push_forward`.

5.2 График

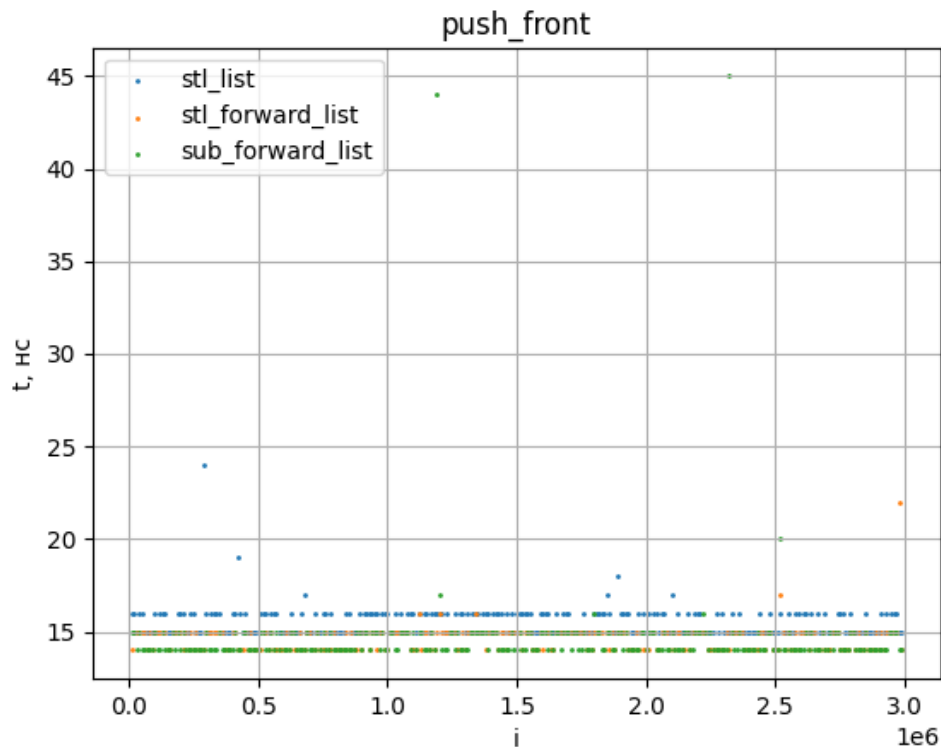


Рис. 4: Среднее время добавления элемента в начало списка

5.3 Асимптотика

Добавление в начало списка требует только создания нового узла и изменения указателя на начало списка. Поэтому сложность операции:

$$T(n) = O(1).$$

6 Эксперимент 4. Удаление элемента из начала односвязного списка

6.1 Описание эксперимента

В данном эксперименте измеряется среднее время удаления элемента из начала списка для контейнеров:

- `std::list`;
- `std::forward_list`;
- пользовательского `subforward_list`.

Для стандартных контейнеров используется метод `pop_front`. Для пользовательского контейнера используется метод `pop_forward`.

6.2 График

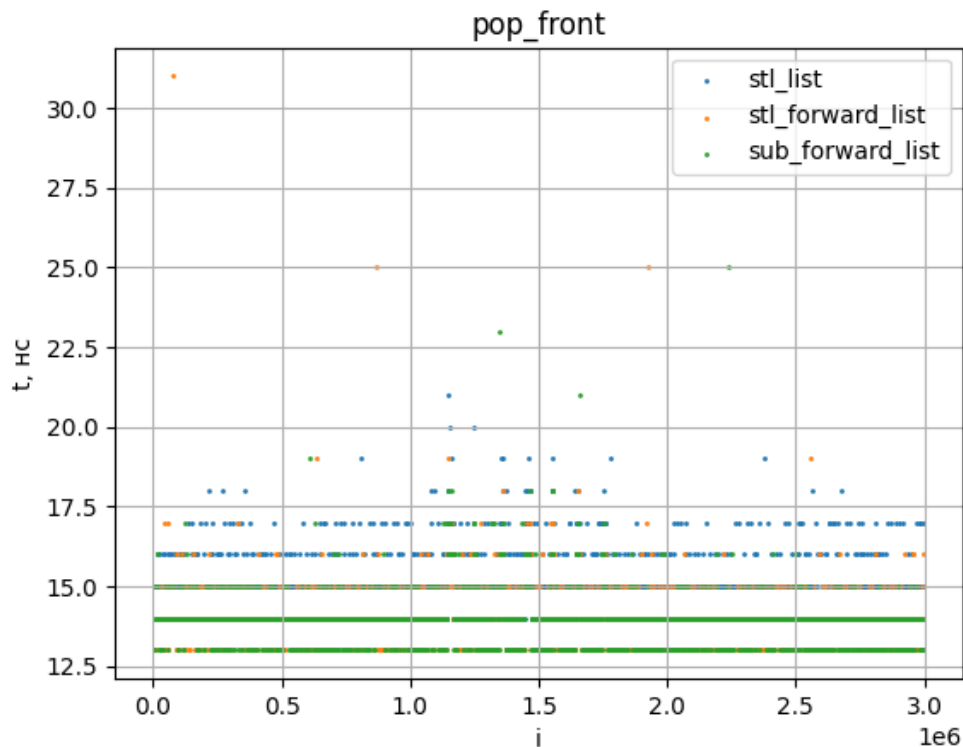


Рис. 5: Среднее время удаления элемента из начала списка

6.3 Асимптотика

Удаление из начала списка требует изменения указателя на первый элемент и освобождения памяти удаляемого узла. Размер списка не влияет на количество выполняемых действий:

$$T(n) = O(1).$$

7 Эксперимент 5. Добавление элемента в бинарное дерево

7.1 Описание эксперимента

В этом эксперименте измеряется среднее время вставки элемента в ассоциативные контейнеры:

- `std::set`;
- `std::map`;
- `std::multiset`;
- `std::multimap`.

7.2 График

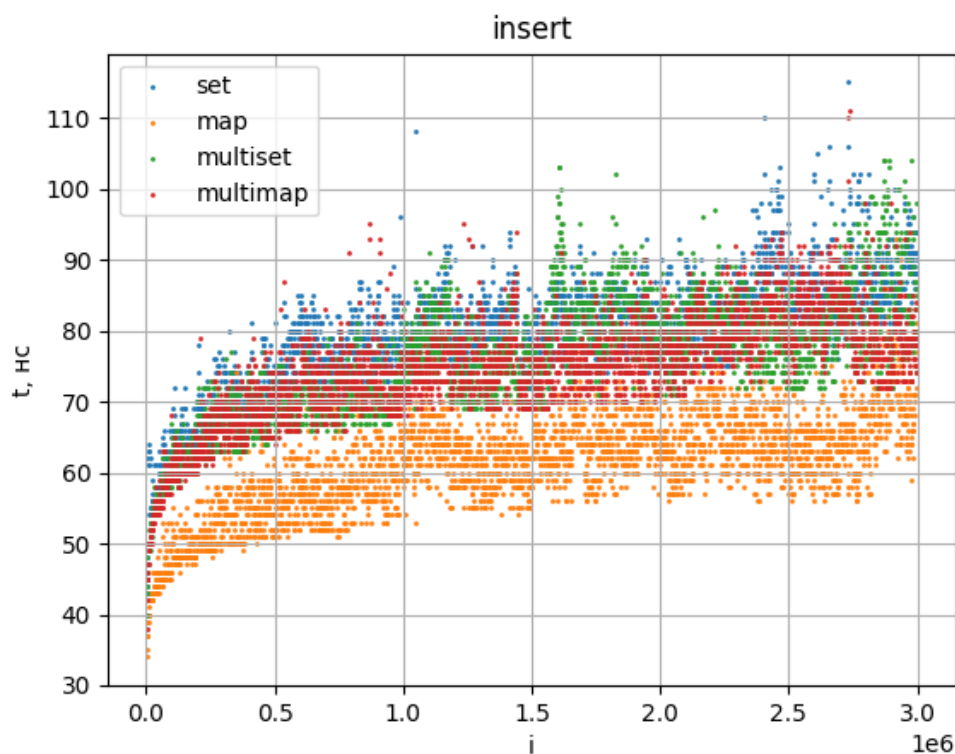


Рис. 6: Среднее время вставки элемента в бинарное дерево

7.3 Асимптотика

Стандартные ассоциативные контейнеры STL обычно реализуются как сбалансированные деревья поиска. Высота такого дерева имеет порядок $\log n$, поэтому вставка элемента выполняется за:

$$T(n) = O(\log n).$$

8 Эксперимент 6. Обход всего контейнера

8.1 Описание эксперимента

В данном эксперименте измеряется среднее время полного обхода контейнеров:

- `std::vector`;
- `std::forward_list`;
- `std::list`;
- `std::map`;
- `std::set`.

В качестве простой операции за $O(1)$ используется арифметическая операция над каждым элементом. Для контейнеров, элементы которых можно изменять, можно увеличивать каждый элемент на единицу. Для `std::set` изменять элементы напрямую нельзя, поэтому для него можно использовать подсчёт контрольной суммы.

8.2 График

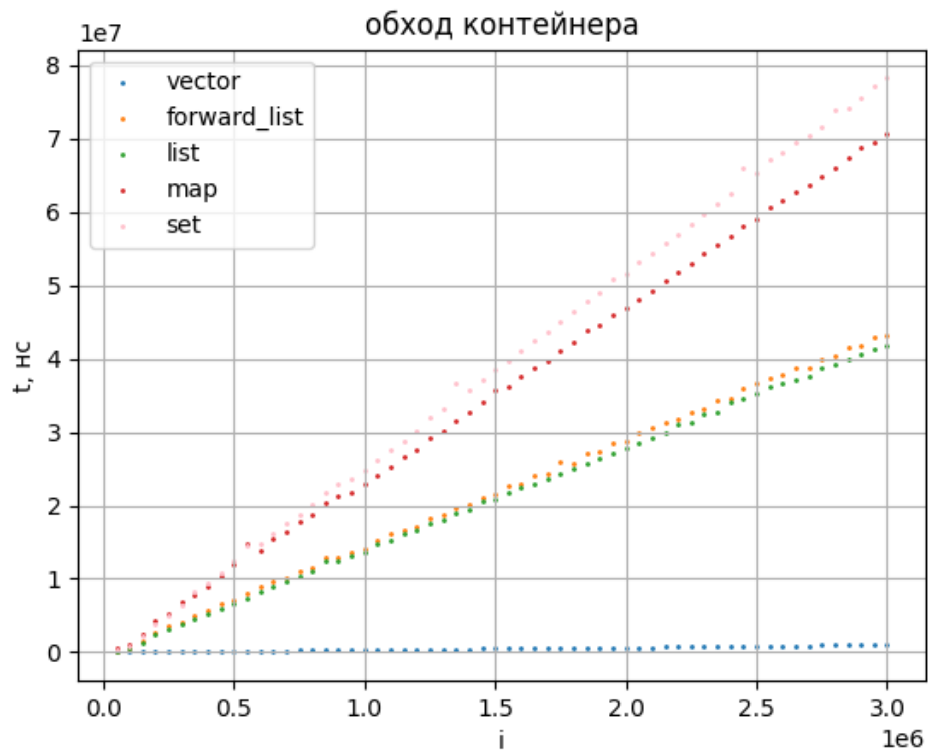


Рис. 7: Среднее время обхода всего контейнера

8.3 Асимптотика

При полном обходе необходимо посетить каждый элемент контейнера. Если обработка одного элемента занимает $O(1)$, то общая сложность:

$$T(n) = O(n).$$

9 Сводная таблица асимптотик

Таблица 1: Теоретические асимптотики исследуемых операций

Операция	Асимптотика
<code>vector::push_back</code> амортизированно	$O(1)$
<code>vector::push_back</code> при реаллокации	$O(n)$
Вставка в произвольное место <code>vector</code> / <code>subvector</code>	$O(n)$
Удаление из произвольного места <code>vector</code> / <code>subvector</code>	$O(n)$
Добавление в начало <code>list</code> , <code>forward_list</code> , <code>subforward_list</code>	$O(1)$
Удаление из начала <code>list</code> , <code>forward_list</code> , <code>subforward_list</code>	$O(1)$
Вставка в <code>set</code> , <code>map</code> , <code>multiset</code> , <code>multimap</code>	$O(\log n)$
Полный обход контейнера	$O(n)$

10 Общий вывод

В ходе лабораторной работы были исследованы временные характеристики основных операций над стандартными и пользовательскими контейнерами C++.

Эксперимент с `std::vector` показал, что его размер увеличивается линейно, а ёмкость — скачкообразно. Это подтверждает механизм резервирования памяти с запасом. Операция `push_back` имеет амортизированную сложность $O(1)$, но отдельные операции реаллокации могут выполняться за $O(n)$.

Вставка и удаление элемента из произвольного места вектора имеют линейную сложность $O(n)$, так как требуют сдвига элементов. Пользовательский контейнер `subvector` должен демонстрировать аналогичную асимптотику при корректной реализации.

Добавление и удаление элемента из начала списков выполняется за $O(1)$, так как для этого достаточно изменить один или несколько указателей. Поэтому графики для `std::list`, `std::forward_list` и `subforward_list` должны быть близки к горизонтальным.

Вставка в ассоциативные контейнеры `std::set`, `std::map`, `std::multiset` и `std::multimap` выполняется за $O(\log n)$, поскольку эти контейнеры основаны на сбалансированных деревьях поиска.

Полный обход любого контейнера требует посещения каждого элемента, поэтому имеет сложность $O(n)$. При этом фактическое время обхода может отличаться из-за особенностей хранения данных в памяти: `std::vector` хранит элементы непрерывно, а списки и деревья используют отдельные узлы, расположенные в разных участках памяти.